



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DIVISION OF COMPUTER SCIENCE

Continuous Machine Learning for Cooperative, Connected and Automated Mobility applications

DIPLOMA THESIS

of

GEORGIOS A. KYRIAKOPOULOS

Supervisor: Panayiotis Tsanakas
Professor

Athens, October 2025



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
DIVISION OF COMPUTER SCIENCE

Continuous Machine Learning for Cooperative, Connected and Automated Mobility applications

DIPLOMA THESIS

of

GEORGIOS A. KYRIAKOPOULOS

Supervisor: Panayiotis Tsanakas
Professor

Approved by the three-member scientific committee on the 30th of October 2025

.....
Panayiotis Tsanakas
Professor NTUA

.....
Andreas-Georgios Stafylopatis
Professor Emeritus NTUA

.....
Georgios Alexandridis
Assistant Professor NKUA

Athens, October 2025

.....

Georgios A. Kyriakopoulos

Graduate of School of Electrical and Computer Engineering,
National Technical University of Athens

Copyright © **Georgios A. Kyriakopoulos, 2025.**

All rights reserved.

Copying, storage, and distribution of this work, in whole or in part, for commercial purposes is prohibited. Reproduction, storage, and distribution for non-profit, educational, or research purposes is permitted, provided that the source is cited and this notice is retained. Inquiries regarding the use of this work for commercial purposes should be directed to the author.

The views and conclusions expressed in this thesis represent those of the author and do not necessarily reflect the official positions of the National Technical University of Athens.

Περίληψη

Λέξεις Κλειδιά:

Abstract

Keywords:

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, **Professor Panayiotis Tsanakas**, for giving me the opportunity to conduct my thesis in a field that has been of great personal interest to me.

I would also like to thank **Dr. Georgios Drainakis** and **Dr. Panagiotis Pantazopoulos** of the **I-SENSE Research Group of ICCS/NTUA** for their valuable feedback and guidance towards the completion of this work.

I am grateful to my family and friends for their support and encouragement throughout this important chapter of my life, especially **Valentina** and **Artemis**, as well as **Nick**, **Apostolis**, **Alex**, **Christini**, and **Elina**.

Finally, and most importantly, I would like to thank **Serafeim** and **George**, with whom I shared the vision, the challenges, and the excitement of this work. Their collaboration and commitment were essential in shaping this thesis, and I truly appreciate sharing this journey with them.

Contents

Contents	11
List of Figures	13
List of Tables	15
Extended Abstract in Greek	17
1 Introduction	19
1.1 Problem Statement	19
1.2 Motivation	19
1.3 Thesis Goal	20
1.4 Contributions	20
1.5 Thesis Structure	21
2 Background	23
2.1 Classical Machine Learning	23
2.1.1 Definition and Learning Paradigms	23
2.1.2 Representative Tabular Models	24
2.1.3 Feature Engineering and Pipelines	24
2.1.4 Bias and Variance, Regularization, and Model Selection	24
2.2 Continuous Learning	25
2.2.1 Motivation and Settings	25
2.3 Evaluation: Metrics and Validation	25
2.3.1 Regression Metrics	25
2.3.2 Cross-Validation and Data Leakage	25
2.4 MLOps: Operating ML Systems	26
2.4.1 Scope and Concerns	26
2.5 Concept Drift	26
2.5.1 Definitions and Taxonomy	26
2.5.2 Drift Detector Families	27
2.5.3 Adaptation Strategies	27
2.6 SUMO Simulator	27
2.6.1 Overview, Purpose, and Capabilities	27
2.6.2 OSM → SUMO Pipeline and Scenario Design	28
2.6.3 Outputs Used in This Work	28
2.7 Software Engineering Technologies	28

2.7.1	Containerization with Docker and Docker Compose	28
2.7.2	Backend	29
2.7.3	Data Validation and Schemas	29
2.7.4	APIs and Service Communication (REST/HTTP)	29
2.7.5	Dashboard	29
2.7.6	Columnar Storage: Apache Parquet	29
2.7.7	Artifacts and Reproducibility	29

Bibliography		33
---------------------	--	-----------

List of Figures

List of Tables

Extended Abstract in Greek

Chapter 1

Introduction

1.1 Problem Statement

Urban mobility environments are highly dynamic and *non-stationary*, meaning that the statistical properties of traffic data change over time. Machine learning models trained on static historical data often assume a fixed data distribution, an assumption that rarely holds in real city conditions [1]. When deployed in a dynamic urban setting, a model’s performance can gradually degrade as driving patterns shift due to *concept drift*, the evolution of the underlying data relationships over time [2]. This drift is triggered by many factors: for example, road network changes (e.g., construction) can alter traffic flow patterns significantly [3]. Even routine variations like weather and seasonal demand swings can lead to abrupt or gradual shifts in vehicle speeds and congestion levels. A model that performs well on dry, low-traffic days may become less accurate during heavy rain or rush-hour peaks if those conditions were not part of its training distribution [4]. the challenge is that static ML models struggle to maintain accuracy in the face of non-stationarity, as their predictive power deteriorates over time unless the models continually adapt to the evolving urban mobility data [1]. Recognizing and addressing this concept drift in cooperative and automated mobility scenarios is therefore a critical problem identified by the research community.

1.2 Motivation

Given the inevitability of concept drift in real-world intelligent transportation systems, there is a clear need for a practical, drift-aware platform tailored to the mobility domain. While concept drift detection and adaptation have been studied in research, existing solutions are often fragmented or confined to laboratory settings [5]. Current tools lack integration of critical components, for instance, traffic simulation, continuous monitoring, and adaptive retraining are rarely all incorporated into one seamless loop. This means practitioners may detect drifts, but they often have no automated way to replay scenarios (e.g. via simulation) or update models on the fly in response to those drifts.

A domain-specific platform would fill this gap by monitoring model performance through time (not just offline metrics), automatically detecting distribution shifts, and triggering model adaptation with minimal human intervention. Such an integrated approach is especially vital in *Cooperative, Connected and Automated Mobility (CCAM)* applications where conditions can change rapidly

and safety is paramount. One key use case is *Estimated Time of Arrival (ETA)* prediction for vehicles, a core service in smart mobility that influences traffic management, logistics, and traveler information [6]. However, many current mobility systems do not continuously recalibrate ETA models as new data (e.g., live traffic feeds or weather updates) arrive. This motivates the creation of a continuous learning platform for mobility: one that integrates traffic simulation (to test “what-if” scenarios and generate synthetic data), live performance dashboards (to observe errors creeping up over time), drift detection algorithms (to raise alerts when the model starts faltering), and an automated retraining pipeline (to deploy updated models when needed). By addressing these needs, we aim to ensure that mobility prediction models remain accurate and robust despite the ever-changing urban environment.

1.3 Thesis Goal

The objective of this thesis is to design, implement, and evaluate a microservice-based platform for *continuous machine learning* in urban mobility applications. In essence, the goal is to create a *drift-aware ML platform* that supports end-to-end monitoring and adaptation of predictive models in a city traffic context.

The platform will be built using a modular microservice architecture to ensure scalability and flexibility, where each component (orchestration, model serving, drift detection, etc.) operates as an independent service. Crucially, the platform is intended to be model-agnostic and domain-specific: it can host any machine learning model for mobility prediction tasks, but this thesis will demonstrate and evaluate it primarily through the lens of an Estimated Time of Arrival (ETA) prediction use case. We focus on ETA because of its importance in CCAM scenarios (helping vehicles and travelers coordinate effectively), and because it provides a tangible benchmark to test how well the platform handles concept drift. The platform will ingest real-time or simulated vehicle telemetry and output ETA predictions, all the while monitoring for performance drops. Upon detecting drift, it will automatically trigger model adaptation (retraining or model swapping) to self-correct the predictions.

Although ETA is the main case study, the system is designed to accommodate additional models, for example, external predictive modules for Fuel Consumption [7] or Number of Stops [8] will be integrated to showcase that multiple concurrent mobility models can coexist and benefit from the shared drift-aware infrastructure. By the end of this thesis, we aim to show that such a platform can be realized and that it effectively maintains model performance over time in a realistic urban traffic setting.

1.4 Contributions

This thesis makes several contributions to the state of the art in continuous machine learning for mobility:

- **Synthetic Dataset Pipeline:** We developed a reproducible, modular data pipeline that generates training and evaluation datasets using synthetic traffic data from Simulation of Urban MObility (SUMO) [9]. Focusing on a case study of central Athens, the pipeline takes an open-source city road network and creates realistic traffic scenarios (vehicles, routes, events) to produce rich labeled data. This approach ensures that experiments are repeatable and tunable (e.g. the pipeline can be re-run for any city region or traffic pattern by adjusting parameters), addressing the shortage of standard drift benchmarks in urban mobility.

- **ETA Prediction Model under Drift:** We designed a feature-engineered ETA prediction model using gradient-boosted decision trees (LightGBM) [10]. The model is trained on the above dataset to estimate travel times for vehicles, and we introduce domain-specific features to improve accuracy. More importantly, the model’s performance is evaluated under simulated drift scenarios (e.g., a sudden shift from normal traffic to heavy rain conditions that reduce vehicle speeds). This allowed us to study how concept drift affects ETA prediction accuracy, and to validate that our model (and platform) can detect and respond to these changes.
- **Time-Lapse Drift Simulation:** We conducted a time-lapse simulation experiment to rigorously assess drift detection and adaptation in an end-to-end fashion. In this setup, the SUMO simulator replays an extended sequence where an initial period of stable conditions is followed by a clear drift event (a transition from dry weather to a storm causing slower traffic). The platform’s drift detectors monitor the real-time prediction error and successfully detect the concept drift when the environment changes. We then measure how the system’s automatic adaptation (triggering model retraining with new data from the rainy period) helps recover the model’s performance. This experiment provides a realistic validation of the platform’s ability to observe, detect, and mitigate drift in a controlled yet lifelike scenario.
- **Drift-Aware Platform Implementation:** We present the design and implementation of a platform for continuous learning in mobility, which is a key practical contribution of this work. The platform includes components for simulation replay (feeding historical or synthetic data through the model as if in real time), live dashboarding of model performance metrics (so that users can visualize concept drift as it happens), pluggable drift detection algorithms (with calibration tools to set detection thresholds and minimize false alarms), and an automatic retraining and deployment pipeline (which seamlessly replaces the old model with an updated one when drift is confirmed). The entire system is built with modern software frameworks (containerized microservices, REST APIs for model serving, and event-driven triggers for retraining) to ensure it can be deployed in real operational environments. By open-sourcing the platform and detailing its architecture, we enable other researchers and practitioners to reproduce our approach and extend it to their own cooperative and automated mobility applications.

1.5 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Background** - Introduces the foundational concepts and tools underlying our work. We review machine learning basics relevant to continuous model updating, the functionalities of the SUMO traffic simulator (for creating mobility scenarios), the notion of concept drift (types of drift and common detection methods), and the platform technologies (microservices, Docker, APIs, etc.) used to build our solution.
- **Chapter 3: Related Work** - Surveys existing literature and systems in areas pertinent to our thesis. We cover drift detection tools and frameworks (e.g. streaming ML libraries and MLOps solutions for model monitoring), examine available urban mobility simulation environments used to study drift, and review prior research on ETA prediction models in intelligent transportation. This chapter highlights how our approach compares to and

innovates upon the current state of the art.

- **Chapter 4: Dataset and Modeling** - Details our methodology for creating the dataset and developing the ETA prediction model. We describe the data generation process using SUMO for the Athens case study, including how traffic demand patterns and drift scenarios (like weather changes) are configured. We then present the ETA model development, including feature engineering, hyperparameter tuning, model selection (why LightGBM) and the overall training procedure and the set up of the experiments.
- **Chapter 5: Platform Architecture** - Provides a comprehensive overview of the drift-aware platform's design. We enumerate the microservices and their roles, explain how the system ingests data (simulation or extensions to real streams), how drift detection algorithms are integrated, and how the platform orchestrates retraining and deployment of models. Implementation details of key components (such as the real-time dashboard) are given to illustrate how the system works in practice.
- **Chapter 6: Results** - Presents the results of our evaluation. We report the baseline performance of the ETA model under static conditions and then analyze its behavior under induced drift (e.g., performance drop during the simulated rain scenario). We demonstrate the effectiveness of drift detection (e.g., detection delay) and quantify the gains from automatic adaptation (improvement in error after retraining). The results validate the platform's ability to maintain model accuracy over time.
- **Chapter 7: Conclusions** - Summarizes the contributions and findings of the thesis. We reflect on the success of the continuous learning approach for CCAM applications and any limitations observed. Finally, we outline future research directions, such as improvements to platform scaling and monitoring, or integrating the system with real-time data streams.

Chapter 2

Background

This chapter develops the theoretical and technological background that underpins the thesis. We begin with classical machine learning (paradigms, representative models, and feature pipelines), then introduce continuous/continual learning and evaluation practice, such as metrics and cross-validation. We next cover MLOps concerns for operating ML systems, followed by concept drift definitions, detector families, and adaptation strategies. We then present the SUMO simulator, its scope, scenario construction, OSM→SUMO pipeline, and outputs used, and conclude with the software engineering stack, including Docker/Compose, FastAPI, Dash, Parquet, and REST/HTTP.

2.1 Classical Machine Learning

2.1.1 Definition and Learning Paradigms

Machine Learning (ML) studies algorithms that improve performance on a task T with experience (data) and a performance measure P [11, 12].

Canonical paradigms include:

- **Supervised learning** (labeled examples $\{(x_i, y_i)\}$): learns $f : x \mapsto y$ for regression (e.g., ETA prediction) or classification (e.g., incident detection). Training typically minimizes an empirical loss (e.g., MAE/MSE or cross-entropy) plus regularization to control model capacity and reduce overfitting. For tabular problems, tree ensembles (e.g., gradient boosting) are strong baselines; for spatiotemporal or image data, representation-learning approaches may be preferable.
- **Unsupervised learning** (unlabeled x): discovers structure for exploration, compression, anomaly detection, or feature construction. Typical families include clustering (e.g., k -means), density modeling (e.g., Gaussian mixtures), and dimensionality reduction (e.g., PCA). These can act as pretext steps that enrich supervised models with informative features.
- **Reinforcement learning** (sequential decision-making): learns policies that maximize cumulative reward in an environment. While not central here, RL is relevant to adaptive traffic signal control and routing policies under uncertainty.

2.1.2 Representative Tabular Models

For tabular, heterogeneous, and mixed-scale features common in mobility datasets:

- **Linear models** (OLS, Lasso/Ridge/ElasticNet): as strong baselines; interpretable coefficients and fast training.
- **k-Nearest Neighbors**: non-parametric, local averaging, simple but memory and latency heavy.
- **Decision trees**: axis-aligned splits, handle nonlinearity and interactions.
- **Random Forests**: bagged trees, robust, low-variance and good baseline for tabular.
- **Gradient-Boosted Trees** (XGBoost [13], LightGBM [10], CatBoost [14]): state-of-the-art on many tabular tasks due to additive boosting of weak learners and strong handling of heterogeneity and monotonic constraints.
- **Neural networks** (MLPs): for high-capacity function approximation with feature learning, often require more careful regularization on small tabular sets.

Model choice depends on data size, feature types, latency budgets, and interpretability requirements.

2.1.3 Feature Engineering and Pipelines

End-to-end *pipelines* ensure the same preprocessing is applied in training and inference, preventing leakage and improving reproducibility [15]. Typical steps include:

- **Cleaning**: schema validation, missing-value imputation, outlier handling.
- **Encoding**: one-hot/ordinal for categoricals, frequency/target encoding with care.
- **Scaling**: standardization/robust scaling when models are scale-sensitive.
- **Feature construction**: e.g., temporal (hour-of-day, day-of-week), spatial (segment length, bearings), network features (degree/centrality proxies), and aggregates over short histories.
- **Dimensionality reduction** (PCA) or feature selection for efficiency and generalization.

Packaging these with a library (e.g., scikit-learn **Pipeline**) produces deterministic training and portable inference graphs [16].

2.1.4 Bias and Variance, Regularization, and Model Selection

Practical supervised learning balances *underfitting* (high bias) and *overfitting* (high variance). Core mechanisms include:

- **Regularization**: penalties (e.g., L1/L2), early stopping, dropout, and tree constraints (depth, leaves, learning rate) to control capacity.
- **Model selection**: hyperparameter tuning via cross-validation on training data only; reserve a final hold-out or temporally later set for unbiased estimates.
- **Baselines and sanity checks**: compare to simple heuristics (e.g., historical medians), verify monotonic effects and feature importances, and perform ablations to confirm signal sources.

These practices, combined with leakage-aware pipelines, produce models that generalize and remain interpretable in operational contexts.

2.2 Continuous Learning

2.2.1 Motivation and Settings

Real-world mobility is *non-stationary*: demand cycles, policy changes, incidents, and weather shift the data distribution over time. **Online/continual learning** updates models incrementally, on rolling windows, or via scheduled retraining to preserve performance under change [17, 18]. Practical axes:

- **Update granularity**: per-instance online updates vs. mini-batch vs. periodic retrains.
- **Forgetting mechanism**: sliding windows, decay weights, or explicit resets.
- **Latency/compute budget**: balancing freshness with cost and downtime.

Tooling such as *River* supports streaming estimators, metrics, and change detectors with low-latency, incremental APIs [5].

Algorithms and tooling. In addition to fully online learners, many scikit-learn estimators expose `partial_fit` or warm-start capabilities suitable for mini-batch updates. River provides native streaming models and metrics. Label delay is common in operations, when labels are unavailable, unsupervised monitors on inputs or prediction distributions can provide early signals until supervised error-based detectors activate.

2.3 Evaluation: Metrics and Validation

2.3.1 Regression Metrics

For ETA and related regression tasks:

- **MAE**: $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ (target units). Interpretable and robust to outliers compared to MSE, reads as the average absolute miss in seconds [19].
- **MAPE**: $\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}$. Scale-free percentage error; unstable when $y \approx 0$ [20].
- **RMSE**: $\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ (target units). More sensitive to large errors than MAE due to squaring; penalizes outliers heavily.
- **R^2** (coefficient of determination): measures proportion of variance explained by the model, ranging from $-\infty$ to 1; values near 1 indicate strong fit, while $R^2 \leq 0$ means the model performs worse than a constant baseline.

Metric choice reflects operational meaning (e.g., a small absolute error in seconds may be more meaningful than a small percentage error during light traffic) and comparability across scenarios.

2.3.2 Cross-Validation and Data Leakage

Generalization is estimated with cross-validation and matched test scenarios:

- **KFold CV**: partition into k folds, rotate the validation fold, and average scores across iterations [21].
- **StratifiedKFold**: preserves target distribution across folds [22].
- **Temporal splits**: chronological splits for time-ordered evaluation (reported where applicable) to avoid look-ahead.

Preprocessing (e.g., scalers/encoders) is fitted within each training fold and applied to its validation fold; features that would introduce future information are excluded, following standard leakage-avoidance practice [15].

2.4 MLOps: Operating ML Systems

2.4.1 Scope and Concerns

MLOps extends DevOps with data and model-centric practices to make ML systems reliable, reproducible, and observable in practice. Core concerns are [23]:

- **Data management**: explicit schemas, validation checks, drift monitors, and lineage across simulation outputs and features.
- **Experiment tracking & model registry**: hyperparameters, metrics, artifacts, and model versions promoted through stages (dev/staging/prod).
- **CI/CD for ML**: automated training and packaging, reproducible container builds, and controlled rollouts.
- **Monitoring**: service SLOs (latency, availability), data and feature quality, prediction and error distributions, drift signals, and alerting.
- **Governance**: reproducibility, audits, access control to artifacts, and explicit rollback plans.

These address well-documented ML “technical debt” risks (glue code, pipeline entanglement, hidden feedback loops) by favoring modular components, versioned artifacts, and automated checks [23].

2.5 Concept Drift

2.5.1 Definitions and Taxonomy

Concept drift is a change in the joint distribution $P(X, Y)$ over time that invalidates a model trained on historical data [1, 2]. Two major forms include:

- **Virtual/data drift**: $P(X)$ changes while $P(Y | X)$ remains stable (covariate shift).
- **Real/concept drift**: $P(Y | X)$ itself changes (the mapping from features to target shifts).

Temporal archetypes include:

- **Sudden/abrupt**: near-instant change (e.g., road closure).
- **Gradual/incremental**: progressive replacement over time (e.g., evolving driver behavior).
- **Recurring/seasonal**: concepts reappear (e.g., weekday vs. weekend regimes, seasons).

2.5.2 Drift Detector Families

Error-based detectors (supervised). Monitor a streaming error signal (e.g., $|\hat{y} - y|$) and trigger when the mean shifts:

- **Page–Hinkley (PHT):** cumulative sum test sensitive to mean increases [24].
- **DDM/EDDM:** compares on-line error to reference to detect rising error rates (classification focus).

Adaptive window tests. **ADWIN** maintains a dynamically sized window. It repeatedly splits into left/right subwindows and applies a statistical test. If means differ beyond a bound, it cuts the window and signals change [25].

Distributional tests (unsupervised or semi-supervised). Compare recent vs. reference input distributions; e.g., **KSWIN** applies Kolmogorov–Smirnov tests to sliding windows to detect significant changes [26]. These are useful when labels are delayed or unavailable.

Some key trade-offs when choosing a drift detector include sensitivity vs. false alarms, label availability (supervised vs. unsupervised), latency, and computational costs.

2.5.3 Adaptation Strategies

Once drift is suspected/confirmed:

- **Retrain (reset):** fit a fresh model on a recent window of data, good for abrupt regime shifts.
- **Incremental updates:** continue training or add estimators (e.g., more boosting rounds) with recency weighting or forgetting.
- **Windowing:** keep a moving window or buffer to emphasize recent data distribution.
- **Ensembles/repositories:** maintain experts specialized to regimes (e.g., baseline model vs. model for rainy conditions); weight or switch based on current evidence; reuse prior specialists when regimes recur.
- **Safeguards:** trigger throttling, confidence thresholds, or fallbacks (e.g., historical medians) during adaptation.

The choice of adaptation strategy depends on factors like drift speed, severity, data availability, compute budgets, and risk tolerance [1, 2].

2.6 SUMO Simulator

2.6.1 Overview, Purpose, and Capabilities

Eclipse SUMO (Simulation of Urban MObility) is an open-source, microscopic, multi-modal traffic simulator developed by the German Aerospace Center (DLR) and now under the Eclipse ecosystem. It is widely used in research and practice for modeling vehicle-level dynamics, routing, signals, and emissions across large networks [27, 28]. “Microscopic” means each vehicle is an agent governed by car-following (for example, the Krauss model [29]) and lane-changing models, enabling fine-grained phenomena, such as queues, spillback and shockwaves.

Key capabilities include:

- **Network handling:** import, edit, and simulate large road graphs with multiple lane attributes, priorities, and traffic signal programs.
- **Demand modeling:** trips, routes, O/D flows, public transport; dynamic routing and rerouting.
- **Control & APIs:** TraCI [30] (TCP control interface) for online interaction; Python clients to step the simulator, inject vehicles, or read states.
- **Scenarios:** controlled experiments (e.g., incidents, weather surrogates, signal changes) with reproducibility via configuration files.

2.6.2 OSM → SUMO Pipeline and Scenario Design

A typical path to realistic scenarios is converting **OpenStreetMap** (OSM) into a SUMO network:

1. Obtain OSM data for the area of interest.
2. Convert with **netconvert** (infers edges, lanes, priorities, optionally traffic lights) [31, 32].
3. Refine with **netedit** (GUI) for manual adjustments and optionally use **polyconvert** for shapes or polygons.
4. Generate demand via **randomTrips.py**, route assignment, or O/D matrices.

This pipeline yields geographically faithful networks (e.g., central Athens) while preserving control over demand, signal plans, and experiment design [28].

2.6.3 Outputs Used in This Work

SUMO produces multiple outputs consumable by analytics/ML:

- **Floating Car Data (FCD):** per-step vehicle states (id, position, speed, angle, lane, etc.) via `--fcd-output`. FCD resembles GPS traces and is well-suited for trip assembly and ETA features [33].
- **Emissions/Fuel** via `--emission-output`: per-vehicle per-step estimates for fuel consumption and pollutants, with documented units and models [34].
- Additional: **tripinfo** (per-trip travel times), edge/lane aggregations, detector outputs, and more [28].

Outputs are XML by default, with support for additional formats like CSV and Parquet available. The XML format can be converted to CSV for efficient downstream processing, with provided tools like `xml2csv`.

2.7 Software Engineering Technologies

2.7.1 Containerization with Docker and Docker Compose

Docker packages code, runtime, and dependencies into portable container images that run consistently across environments [35, 36]. The benefits of using it include:

- Reproducible builds and isolated dependencies.

- Fast startup and easy horizontal scaling via replicas.
- Immutable artifacts that enable safe rollbacks.

Docker Compose declaratively defines multi-container applications (services, networks, volumes) and brings them up with a single command, ideal for local development or single-host deployments of microservices [35].

2.7.2 Backend

FastAPI is a high-performance Python framework for REST APIs with type-hinted models, validation (Pydantic [37]), and autogenerated OpenAPI/Swagger docs [38]. It suits prediction endpoints, orchestration commands (e.g., retraining triggers), and health checks.

2.7.3 Data Validation and Schemas

Schemas enforced at API boundaries and in dataframes reduce runtime errors and prevent silent corruption. Typed request/response models (e.g., Pydantic) and dataframe-level checks (e.g., declared dtypes, value ranges, categorical domains) provide early failure and clearer contracts between services.

2.7.4 APIs and Service Communication (REST/HTTP)

Services interact over **HTTP** following **REST** principles (uniform interface, statelessness, resource orientation) [39]. Conventions include:

- Clear resource paths (e.g., `/predict`, `/models/current`, `/drift/status`).
- JSON payloads with explicit schemas; versioned endpoints.
- Proper status codes (2xx success, 4xx client errors, 5xx server errors) and idempotency where applicable.

Within Docker networks, services resolve each other by name; an API gateway or reverse proxy can front external access. OpenAPI contracts facilitate testing and integration.

2.7.5 Dashboard

Dash builds interactive dashboards in Python using Plotly components, enabling real-time monitoring (metrics, error traces, drift alarms) and control widgets (e.g., start/stop simulation) without JavaScript [40].

2.7.6 Columnar Storage: Apache Parquet

Parquet is a columnar format optimized for analytical IO: selective column reads, strong compression, and explicit schemas. It integrates with PyArrow and Pandas and supports large, append-friendly datasets typical in streams and simulations [41, 42].

2.7.7 Artifacts and Reproducibility

Models and datasets should be versioned and stored as immutable artifacts with metadata (schema, training data window, metrics). Deterministic serialization (e.g., joblib) and pinned

environments support reproducible inference; storing artifacts in object storage enables auditing and rollback.

Bibliography

- [1] Gama, J., Žliobaite, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. “A Survey on Concept Drift Adaptation”. In: *ACM Computing Surveys* 46.4 (2014), pp. 1–37. DOI: 10.1145/2523813. URL: <https://dl.acm.org/doi/10.1145/2523813>.
- [2] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. “Learning under Concept Drift: A Review”. In: *IEEE Transactions on Knowledge and Data Engineering* (2018). DOI: 10.1109/TKDE.2018.2876857.
- [3] Webb, G. I., Hyde, R., Cao, H., Nguyen, H., and Petitjean, F. “Characterizing concept drift”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 1285–1294.
- [4] Ross, G. J. “Exponential smoothing for concept drift adaptation”. In: *Expert Systems with Applications* 40.6 (2013), pp. 2006–2016.
- [5] Montiel, J., Halford, M., Mastelini, S. M., Bolmier, G., Sourty, R., Vaysse, R., Zouitine, A., Guillet, F., Read, J., Abdessalem, T., and Bifet, A. “River: Machine Learning for Streaming Data in Python”. In: *Journal of Machine Learning Research* 22.110 (2021), pp. 1–8. URL: <https://www.jmlr.org/papers/v22/20-1380.html>.
- [6] Li, Y. and Rose, G. “Short-term travel time prediction on freeway segments based on adaptive Kalman filter”. In: *Transportation Research Record* 2165.1 (2012), pp. 69–78.
- [7] Angelis, G. *Fuel Consumption Prediction Model*. Unpublished work, integrated in drift-aware platform. 2025.
- [8] Tzelepis, S. *Number of Stops Prediction Model*. Unpublished work, integrated in drift-aware platform. 2025.
- [9] Krajzewicz, D., Erdmann, J., Behrisch, M., and Bieker, L. “Recent development and applications of SUMO - Simulation of Urban MObility”. In: *International Journal On Advances in Systems and Measurements* 5.3&4 (2012), pp. 128–138.
- [10] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. 2017, pp. 3146–3154. URL: <https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>.
- [11] Murphy, K. P. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012. URL: <https://www.cs.ubc.ca/~murphyk/MLbook/pml-toc-1may12.pdf>.
- [12] Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. URL: <https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/>.
- [13] Chen, T. and Guestrin, C. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: <https://arxiv.org/abs/1603.02754>.

- [14] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. “CatBoost: Unbiased Boosting with Categorical Features”. In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. 2018, pp. 6638–6648. URL: <https://arxiv.org/abs/1706.09516>.
- [15] *Metrics and scoring: quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed 2025-10-25.
- [16] *Pipeline: chaining estimators — scikit-learn*. <https://scikit-learn.org/stable/modules/compose.html>. Accessed 2025-10-25.
- [17] Shalev-Shwartz, S. “Online Learning and Online Convex Optimization”. In: *Foundations and Trends in Machine Learning* 4.2 (2012), pp. 107–194. DOI: 10.1561/22000000018. URL: <https://www.cs.huji.ac.il/~shais/papers/OLsurvey.pdf>.
- [18] Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. “Continual Lifelong Learning with Neural Networks: A Review”. In: *Neural Networks* 113 (2019), pp. 54–71. DOI: 10.1016/j.neunet.2019.01.012. URL: https://www2.informatik.uni-hamburg.de/wtm/publications/2019/PKPKW19/Parisi-2019_Continual-lifelong-learning-with-NN-review.pdf.
- [19] *mean_absolute_error — scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html. Accessed 2025-10-25.
- [20] *mean_absolute_percentage_error — scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_percentage_error.html. Accessed 2025-10-25.
- [21] *Cross-validation: evaluating estimator performance — scikit-learn*. https://scikit-learn.org/stable/modules/cross_validation.html. Accessed 2025-10-25.
- [22] *StratifiedKfold — scikit-learn*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKfold.html. Accessed 2025-10-25.
- [23] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. “Hidden Technical Debt in Machine Learning Systems”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015, pp. 2503–2511. URL: <https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>.
- [24] Page, E. S. “Continuous Inspection Schemes”. In: *Biometrika* 41.1-2 (1954), pp. 100–115. DOI: 10.1093/biomet/41.1-2.100. URL: <https://academic.oup.com/biomet/article-abstract/41/1-2/100/456627>.
- [25] Bifet, A. and Gavaldà, R. “Learning from Time-Changing Data with Adaptive Windowing”. In: *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*. 2007, pp. 443–448. DOI: 10.1137/1.9781611972771.42. URL: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611972771.42>.
- [26] Raab, C., Heusinger, M., and Schleif, F.-M. “Reactive Soft Prototype Computing for Concept Drift Streams”. In: *Neurocomputing* 398 (2020). KSWIN, pp. 309–327. DOI: 10.1016/j.neucom.2020.02.119. URL: <https://arxiv.org/pdf/2007.05432>.
- [27] Alvarez Lopez, P., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flöttmann, Y.-P., Hilbrich, A., Lücken, L., Rummel, J., Wagner, P., and Wießner, E. “Microscopic Traffic Simulation using SUMO”. In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. 2018. DOI: 10.1109/ITSC.2018.8569938. URL: <https://elib.dlr.de/127994/1/08569938.pdf>.
- [28] *Eclipse SUMO Documentation (Overview/Output)*. <https://sumo.dlr.de/docs/>. Accessed 2025-10-25.

-
- [29] Krauß, S. “Microscopic modeling of traffic flow: Investigation of collision free vehicle dynamics”. PhD thesis. DLR, 1998.
 - [30] Wegener, A., Piórkowski, M., Raya, M., Hellbrück, H., Fischer, S., and Hubaux, J.-P. “TraCI: an interface for coupling road traffic and network simulators”. In: *Proceedings of the 11th Communications and Networking Simulation Symposium*. 2008, pp. 155–163.
 - [31] *netconvert — SUMO*. <https://sumo.dlr.de/docs/netconvert.html>. Accessed 2025-10-25.
 - [32] *Networks: Import from OpenStreetMap — SUMO*. <https://sumo.dlr.de/docs/Networks/Import/OpenStreetMap.html>. Accessed 2025-10-25.
 - [33] *Simulation Output: FCD Output — SUMO*. <https://sumo.dlr.de/docs/Simulation/Output/FCDOutput.html>. Accessed 2025-10-25.
 - [34] *Simulation Output: EmissionOutput — SUMO*. <https://sumo.dlr.de/docs/Simulation/Output/EmissionOutput.html>. Accessed 2025-10-25.
 - [35] *What is Docker?* <https://docs.docker.com/get-started/overview/>. Accessed 2025-10-25.
 - [36] *What is a container?* <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>. Accessed 2025-10-25.
 - [37] *Pydantic: Data validation and settings management using Python type hints*. Version 2.x, Available: <https://docs.pydantic.dev/latest/>. 2025.
 - [38] Ramírez, S. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>. Accessed 2025-10-25.
 - [39] Fielding, R. T. “Architectural Styles and the Design of Network-based Software Architectures”. PhD thesis. University of California, Irvine, 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf.
 - [40] *Dash Documentation & User Guide*. <https://dash.plotly.com/>. Accessed 2025-10-25.
 - [41] *Apache Parquet Documentation*. <https://parquet.apache.org/docs/>. Accessed 2025-10-25.
 - [42] *Reading and Writing the Apache Parquet Format (PyArrow)*. <https://arrow.apache.org/docs/python/parquet.html>. Accessed 2025-10-25.